B-trees

Part V Advanced Data Structures

Scope

Packing multiple keys into a page reduces disk-page search paths. Insertion and deletion must move child pointers with keys to preserve global order, occupancy bounds, and equal leaf depth. Page I/O and within-page CPU work are counted separately.

Prerequisites

Multiway search trees, disk I/O, and height balance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Define node capacity using minimum degree t.
- Analyze search, splits, and merges in page I/O.
- Check root and key-count boundaries.
- 18.1 Definition of B-trees
- 18.2 Basic operations on B-trees
- 18.3 Deleting a key from a B-tree

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Definition of B-trees

A node stores multiple sorted keys and child pointers, typically in one disk page. Keys are distinct; associated data or record pointers move with each key. Duplicate insertion reports EXISTS. This B-tree stores data in internal nodes, unlike a B+ tree with leaf-only data and copied parent separators.

Minimum degree t is an integer≥2. Every nonroot holds t−1..2t−1 keys; a nonempty root holds1..2t−1. An internal node with k keys has exactly k+1 children. All leaves have depth h, with root depth0.

Keys a0<…<a_(k−1) partition global child ranges: child0 keys<a0, child i keys in(a_(i−1),a_i), child k keys>a_(k−1). Sorted keys within each node alone do not prove these subtree bounds.

An empty tree is a zero-key leaf root at height0. The height theorem below assumes n≥1.

Upper and lower height bounds

A minimum-occupancy nonempty height-h tree has one root key and two children; other nodes have t−1 keys. Depth i≥1 contains at least2t^(i−1) nodes, so

$n \geq 1+\Sigma_{i=1}^{h} 2t^{i-1}(t-1)=2t^h-1$,

hence $h \leq \log_t((n+1)/2)$. At h=0 the sum is empty and the formula still holds.

Conversely, at most2t−1 keys and2t children per node give

$n \leq (2t-1)\Sigma_{i=0}^{h}(2t)^i=(2t)^{(h+1)}-1$,

hence $h \geq \log_{(2t)}(n+1)-1$. These describe sparsest and densest legal trees; they do not predict a typical occupancy.

Page bounds for a billion keys

For t=100,n=10^9, height2 holds at most200^3-1=7,999,999 keys, insufficient. Height3 holds up to200^4-1=1,599,999,999, so minimum possible height is3.

Height4 needs at least2 • 100^4-1=199,999,999 keys and remains possible. Height5 needs19,999,999,999,too many. Thus h∈{3,4},giving4 or5 visited nodes.

With a resident root and no other cached pages, this is3 or4 page reads; without cached root,4 or5. Linear within-page search needs O(t) comparisons,CPU O(th). Binary search reduces search comparisons to O(h log t),but insertion/deletion array movement may still cost O(t) per level.

Page size,key/record/child-pointer sizes,and metadata determine t. I/O counts are not measured disk latencies and cannot be replaced by comparison counts.

Basic search

Use zero-based node arrays. Find the first key≥k,or use key count if none.

```
search(x,k):
    i = first index with x.keys[i]>=k, or len(x.keys)
    if i<len(x.keys) and x.keys[i]==k: return (x,i)
    if x is leaf: return NOT_FOUND
    read page x.children[i]
    return search(x.children[i],k)
```

Without an equal key,the separators leave at most one child that can contain k. All others are excluded by global order. Remaining height decreases at each step,ending at a successful key or unsuccessful leaf. Duplicate checking before insertion adds at most one path and leaves the asymptotic I/O bound unchanged.

Complete relocation when splitting a full child

Assume parent p is not full and y=p.children[i] has2t−1 keys. Save median=y.keys[t−1] before splitting.

```
z.keys = y.keys[t .. 2t-2]
y.keys = y.keys[0 .. t-2]
if y is internal:
    z.children = old y.children[t .. 2t-1]
    y.children = old y.children[0 .. t-1]
p.keys.insert(i,median)
p.children.insert(i+1,z)
write y,z,p
```

Each side has t−1 keys and,if internal,t children. The median moves out of the child into the parent as a real key; it is not duplicated. Original child ranges already lie on the correct sides of the median,so order and leaf depth remain. The nonfull parent can accept one key without exceeding2t−1.

Split before descending to keep the current node nonfull

```
insert(k):
    if search(root,k) succeeds: return EXISTS
    if root is full:
        root = new empty internal node with old root as child0
        split(root,0)
    x=root
    while x is internal:
        i = child index selected by k
        if x.children[i] is full:
            split(x,i)
            if k>x.keys[i]: i++
        x=x.children[i]
    insert k into x.keys in sorted position
```

Invariant:x is nonfull. Splitting a full child before descent yields a chosen t−1-key child,so the next x is also nonfull. Leaf insertion adds at most one key without overflow. At most one split occurs per level. Only root splitting raises height,and it raises every leaf equally.

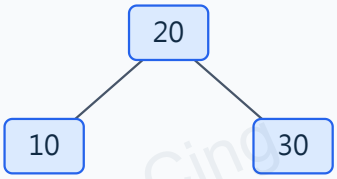
Split before descending to keep the current node nonfull (continued)

Split a full root, then insert25

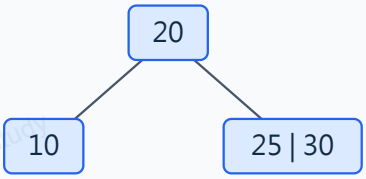
1 Before insertion, t=2



2 Split, before insert25



3 Insert25 on the right



Median20 moves up and leaves the old leaf. Since25>20, insert25 into the right leaf.

Numerical root-split check

At t=2,insert25 into full leaf root[10,20,30].

Stage	Root	Children	Inorder
Before	[10,20,30],leaf	None	10,20,30
Split first	[20]	[10],[30]	10,20,30
Insert25	[20]	[10],[25,30]	10,20,25,30

20 appears only at root.25>20 chooses child1; that leaf grows from1 to2 keys,within maximum3. Both leaves have depth1.

A split writes old child,new child,and parent,a constant number of pages. State whether parent/old child are already cached. O(h) means linear in height,not an identical fixed read/write count for every insertion.

Deleting a key

Use a stronger descent condition: every recursively entered nonroot starts with at least t keys,so losing one leaves at least t−1. At the end,a zero-key internal root may be replaced by its sole child.

When k is found:

1. In a leaf,erase it directly.
2. In an internal node,if its left child has at least t keys,replace k by that subtree's maximum predecessor and delete it there. Otherwise use the successor if the right child has at least t keys.
3. If both children have t−1 keys,merge left child,parent key k,and right child into2t−1 keys,then delete k in the merged node.

Erasing an internal key without adjusting children breaks key/child counts and separators. Predecessor/successor replacement also moves the associated record data.

Repair the target child before descending

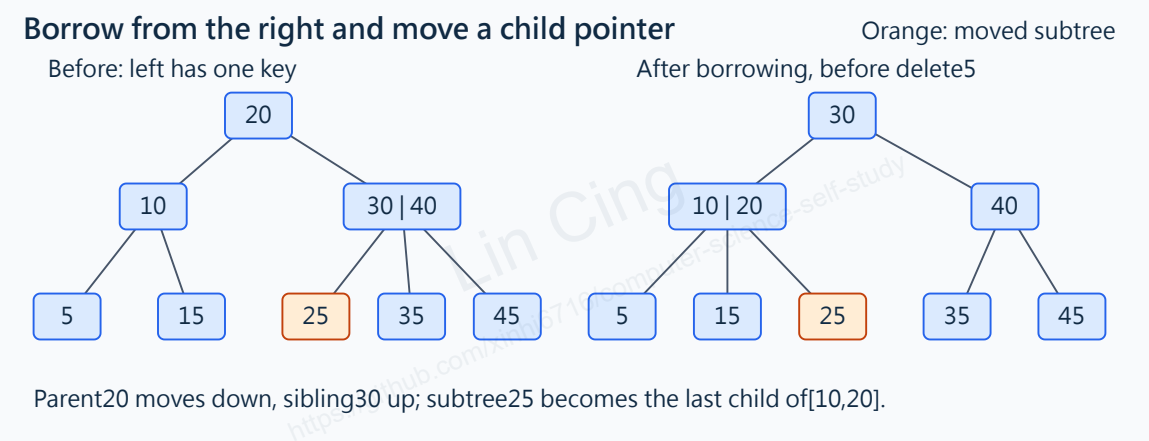
If x lacks k,search order selects child y. Descend directly if y has≥t keys. Otherwise y has t−1:

- With a left sibling holding≥t,move the parent separator to y's front and the sibling's maximum into the parent. For internal nodes,move that sibling's rightmost child to y's first child position.
- Otherwise,with a rich right sibling,move the separator to y's end and that sibling's minimum into the parent. Move its leftmost child to y's last child position.
- Otherwise merge y with an adjacent sibling and their parent separator,yielding2t−1 keys. If merging left,reduce the target child index by1.

Borrowing preserves both children’ s lower bounds without changing parent key count. Merging removes one parent key and one child while preserving ranges and leaf depth. An absent k may still trigger legal restructuring before NOT_FOUND,but the key set must remain unchanged.

Borrowing from the right also moves a child pointer

At t=2,deleting5 starts with a one-key left root child,too small for direct descent,and a two-key right child able to lend.



After borrowing,root30 has left child[10,20] with leaves[5],[15],[25],and right child[40] with[35],[45]. Subtree25 must move:left of new root30 and greater than20,it belongs as[10,20]'s last child.

Borrowing from the right also moves a child pointer (continued)

This is pre-descent repair;5 has not yet been removed. Next,[5] and adjacent[15] both have one key,so merge them with parent10 to[5,10,15],then delete5 to get[10,15]. Left internal node becomes[20],children[10,15],[25]. Root30 and right side remain. All leaves still have depth2.

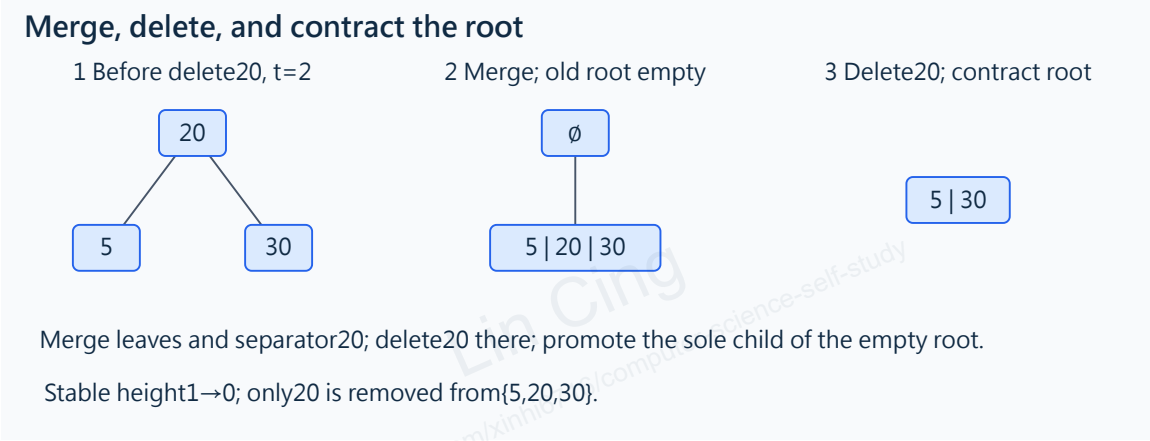
Internal replacement and root contraction

For root[20],left leaf[5,10],right leaf[30],delete20 via predecessor10. Result root[10],left[5],right[30],inorder5,10,30. Do not copy10 upward and forget to remove its old leaf occurrence.

For root[20] with leaves[5],[30],both are minimum-size. Merge to[5,20,30],leaving the old root temporarily empty with one child. Delete20 in the merged leaf to obtain[5,30],then replace the empty internal root by that child.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Internal replacement and root contraction (continued)



Only an empty root may contract this way. Bypassing a general internal node would violate equal leaf depth. Deleting the last key retains a zero-key leaf root as the empty tree.

Deletion pseudocode and induction

```
delete(x,k):
    i = first key >=k, or key count
    if key i equals k:
        if leaf: erase key i; return FOUND
        if child i has >=t keys: replace k by pred; delete(child i,pred)
        elif child i+1 has >=t keys: replace k by succ; delete(child i+1,succ)
        else: y=merge(child i,key i,child i+1); delete(y,k)
        return FOUND
    if leaf: return NOT_FOUND
    y=child i
    if y has t-1 keys: borrow from a rich neighbor, else merge as above
    return delete(the repaired target child,k)
after top call: if root has0keys and1child: root=its child
```

Induct on remaining height. The descent condition lets a leaf lose one key without falling below t−1. Internal replacement preserves order while removing exactly target k from the total set. Borrow/merge gives a legal smaller-height subproblem. A nonroot x starts with≥t keys,so losing one separator in a merge remains legal. Final root contraction handles the exception. This proves termination,key-set correctness,and structural invariants.

Operation costs and implementation checks

Each descent handles constantly many adjacent nodes,with O(1) page reads/writes per level. Search,insertion,and deletion use O(h+1)=O(1+log_t(n+1)) page accesses. Within-node key/child movement costs O(t),giving CPU O(t(h+1)). Finding predecessor/successor separately before deletion adds only a constant number of paths in that subtree; these can also be combined into one descent.

Assume each node fits a page and a fixed root-cache model. Concurrency locks,transaction logs,compression,and fragmentation are additional implementation contracts,not consequences of abstract B-trees.

After each operation,check global key ranges,nonroot occupancy,child count=keys+1,equal leaf depth,no repeated child pointers,and inorder agreement with a reference set. Finite programs catch implementation errors; the induction proves arbitrary legal operations.

Sources

Bibliography

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter18.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study